

Day 1 — Modern JavaScript (ES6+)

Your simple, friendly guide for today

COMING FROM KnockoutJS / .NET → moving to React + TypeScript

TODAY'S GOAL Get comfortable with the everyday modern JavaScript you will use in every React file — explained simply, no rush.

How to use this: First read **Section 1 (Learning Plan)** to see each topic with a small example. Then read **Section 2 (Detailed Notes)** slowly, top to bottom — one read should make everything click. Keep **Section 3 (Common Mistakes)** handy while you practise.

1 Learning Plan

Day 1 - Modern JavaScript ES6+ (Learning Plan, first 2 hours)

Welcome to Day 1. You already know how to program - variables, loops, functions, classes - so we are not teaching programming. We are teaching **modern JavaScript idioms**: the newer, cleaner ways of writing JS that you will see everywhere in React code. Coming from KnockoutJS and .NET, a lot of this will feel familiar in spirit; the syntax is just different. Read each topic, type the snippet yourself (in Node with `node file.js`, or the browser console), and confirm the output.

1. `let` and `const`

In modern JS we no longer use `var`. Use `const` by default (the binding cannot be reassigned) and `let` only when you truly need to reassign. Both are **block-scoped**, meaning they only live inside the nearest `{ }` - much saner than the old function-scoped `var`. Note `const` means the *variable* cannot be reassigned, not that the object is frozen: you can still mutate a `const` object's properties.

```
const name = "Het";
let count = 0;
count = count + 1;      // fine, let allows reassignment
// name = "Other";      // TypeError: cannot reassign a const

const user = { age: 30 };
user.age = 31;          // allowed - we mutated, not reassigned
console.log(name, count, user.age); // Het 1 31
```

Coming from KnockoutJS/.NET: `const` is like C#'s `readonly` reference (the field can't be repointed, but the object it points to is still mutable), and block scoping matches C#'s `{ }` scoping - unlike old `var`.

Expected outcome: You default to `const`, reach for `let` only when reassigning, and never write `var` again.

2. Arrow Functions

Arrow functions are a shorter syntax for writing functions: `(args) ⇒ body`. If the body is a single expression, you can drop the braces and the `return` - it returns that expression automatically. The big behavioral difference from regular functions is that arrows do **not** have their own `this`; they capture `this` from the surrounding scope, which quietly fixes a whole category of callback bugs.

```
const add = (a, b) => a + b;           // implicit return
const square = x => x * x;             // one param, no parens needed
const greet = () => {                 // block body needs explicit return
  const msg = "hi";
  return msg;
};
console.log(add(2, 3), square(4), greet()); // 5 16 hi
```

Coming from KnockoutJS/.NET: Nearly identical to C# lambdas (`(a, b) => a + b`), and the "captures surrounding `this`" behavior spares you the `var self = this;` trick you used in Knockout view models.

Expected outcome: You can write arrow functions with implicit and block bodies and know they inherit `this` from where they are defined.

3. Template Literals

Template literals are strings written with backticks ``` instead of quotes. Inside them you can embed any expression with `${ ... }`, and they can span multiple lines without `\n`. This replaces messy string concatenation with `+`.

```
const name = "Het";
const items = 3;
const line = `Hello ${name}, you have ${items} item${items === 1 ? "" : "s"}.`;
const multi = `Line one
Line two`;
console.log(line); // Hello Het, you have 3 items.
console.log(multi); // prints across two lines
```

Coming from KnockoutJS/.NET: This is C#'s string interpolation (`$"Hello {name}"`) - same idea, just backticks and `${}` instead of `$" ... "` and `{}`.

Expected outcome: You build strings with `${}` interpolation instead of `+` concatenation, including multi-line strings.

4. Destructuring

Destructuring unpacks values out of objects or arrays into standalone variables in one line. For objects you match by property name (`{ a, b }`); for arrays you match by position (`[first, second]`). You can rename, set defaults, and skip elements - it dramatically cuts down repetitive `x = obj.x` lines.

```
const user = { id: 1, name: "Het", role: "dev" };
const { name, role } = user;           // pull out by key
const { role: job = "guest" } = user;  // rename + default

const nums = [10, 20, 30];
const [first, , third] = nums;         // skip the middle
console.log(name, role, job, first, third); // Het dev dev 10 30
```

Coming from KnockoutJS/.NET: Object destructuring is like C# 7+ tuple/positional deconstruction (`var (name, role) = ...`), but here it matches object keys by name, which is even handier for pulling props apart (you will use this constantly for React props).

Expected outcome: You can pull multiple fields out of objects/arrays in one statement, with renaming and defaults.

5. Spread and Rest (...)

The `...` operator does two opposite jobs depending on where it sits. **Spread** expands an array or object into its elements/properties - great for copying and merging immutably. **Rest** collects leftover items into a single array - great for "the remaining arguments" or "everything else in this object."

```
const base = { a: 1, b: 2 };
const extended = { ...base, c: 3 };      // spread: copy + add → {a:1,b:2,c:3}
const merged = [...[1, 2], ...[3, 4]];  // spread arrays → [1,2,3,4]

const [head, ...tail] = [1, 2, 3, 4];   // rest: head=1, tail=[2,3,4]
const sum = (...nums) ⇒ nums.reduce((t, n) ⇒ t + n, 0); // rest params
console.log(extended, merged, tail, sum(1, 2, 3)); // {a:1,b:2,c:3} [1,2,3,4] [2,3,4] 6
```

Coming from KnockoutJS/.NET: Rest parameters (`(...nums)`) are JS's `params` keyword (`params int[] nums`), and spread-to-copy is how you will do immutable updates in React - similar in spirit to creating a new object rather than mutating, which Knockout observables let you avoid thinking about.

Expected outcome: You can copy/merge arrays and objects immutably with spread, and gather variable arguments or "the rest" with rest.

6. Array Methods (map / filter / reduce / find / sort / some / every)

These are the workhorses of modern JS - declarative ways to transform arrays without manual `for` loops.

`map` transforms every item, `filter` keeps items matching a test, `reduce` folds an array into a single value, `find` returns the first match, `some` / `every` return booleans, and `sort` orders items (pass a comparator, and note it mutates in place). Chain them to express data pipelines clearly.

```
const nums = [5, 2, 8, 1, 9];
const doubled = nums.map(n => n * 2);           // [10,4,16,2,18]
const evens = nums.filter(n => n % 2 === 0);     // [2,8]
const total = nums.reduce((sum, n) => sum + n, 0); // 25
const firstBig = nums.find(n => n > 6);          // 8
const anyBig = nums.some(n => n > 6);            // true
const allPositive = nums.every(n => n > 0);      // true
const sorted = [...nums].sort((a, b) => a - b);  // copy first, then [1,2,5,8,9]
console.log(doubled, evens, total, firstBig, anyBig, allPositive, sorted);
```

Coming from KnockoutJS/.NET: This is LINQ. `map` = `Select`, `filter` = `Where`, `reduce` = `Aggregate`, `find` = `FirstOrDefault`, `some` = `Any`, `every` = `All`. Key gotcha: JS `sort` mutates the original array (unlike LINQ's `OrderBy`), so copy with `[...arr]` first.

Expected outcome: You reach for `map` / `filter` / `reduce` / `find` / `some` / `every` instead of manual loops, and you copy before `sort`.

7. Promises

A Promise represents a value that will arrive later - the result of an async operation that is either still pending, fulfilled (success), or rejected (error). You attach `.then()` to handle success and `.catch()` to handle errors, and you can chain `.then()` calls to sequence steps. Promises are the foundation that `async/await` (next topic) is built on.

```
const fetchUser = () =>
  new Promise((resolve, reject) => {
    const ok = true;
    setTimeout(() => (ok ? resolve({ id: 1 }) : reject(new Error("failed"))), 100);
  });

fetchUser()
  .then(user => console.log("got", user.id)) // got 1
  .catch(err => console.log("error", err.message));
```

Coming from KnockoutJS/.NET: A Promise is JS's `Task<T>`. `.then()` is roughly `ContinueWith` / `await` continuation, `resolve` is the returned result, and `reject` throwing is like a faulted Task carrying an exception.

Expected outcome: You can create a Promise, consume one with `.then().catch()`, and reason about pending/fulfilled/rejected states.

8. async / await

`async/await` is cleaner syntax over Promises so async code reads like normal top-to-bottom code. Marking a function `async` makes it return a Promise; inside it, `await` pauses until a Promise settles and gives you the resolved value. Wrap awaits in `try/catch` to handle rejections just like synchronous errors.

```
const fetchUser = () => Promise.resolve({ id: 1, name: "Het" });

async function load() {
  try {
    const user = await fetchUser(); // pauses until resolved
    console.log("loaded", user.name); // loaded Het
  } catch (err) {
    console.log("error", err.message);
  }
}

load();
```

Coming from KnockoutJS/.NET: This is almost line-for-line C#'s `async` / `await` over `Task` - `await` unwraps the value, `try/catch` catches faults - so it should feel immediately natural.

Expected outcome: You can write `async` functions, `await` Promises, and handle errors with `try/catch` instead of `.then/.catch` chains.

9. ES Modules (import / export)

ES Modules let you split code across files and share pieces explicitly. Each file uses `export` to expose values and `import` to pull them in. A file can have many **named** exports (imported by exact name in braces) and one **default** export (imported under any name). This is how every React project organizes components and utilities.

```
// math.js
export const add = (a, b) => a + b; // named export
export default function multiply(a, b) { // default export
  return a * b;
}

// app.js
import multiply, { add } from "./math.js"; // default + named
console.log(add(2, 3), multiply(2, 3)); // 5 6
```

Coming from KnockoutJS/.NET: `import` / `export` are like C# `using` plus `public` visibility, but explicit per-symbol: you choose exactly what a file exposes (named exports) and what its "main" thing is (default export), rather than everything public being globally referenceable via namespaces.

Expected outcome: You can split code into modules, use named and default exports, and import them correctly with the right brace syntax.

Quick recap

By the end of these two hours you should be comfortable reading and writing all nine idioms above. Every one of them shows up on nearly every line of a React + TypeScript codebase, so getting them into muscle memory now makes Days 2-10 far easier. Next up (later today): hands-on exercises applying these together.

Extra Topics — Setup, Tools & Mindset

A quick, practical setup so your tools are ready for the whole 10 days.

1. VS Code setup for JS / React / TS work

Install these extensions now so they're ready for the whole 10 days:

- **ESLint** (`dbaeumer.vscode-eslint`) — inline lint errors.
- **Prettier - Code formatter** (`esbenp.prettier-vscode`) — auto-formatting.
- **JavaScript and TypeScript Nightly** (optional) — latest TS features (useful from Day 2).
- **Path Intellisense** and **npm Intellisense** — import path autocomplete.
- **Error Lens** — shows errors inline on the line (great for a fast feedback loop).

Recommended `settings.json` (workspace `.vscode/settings.json`):

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": { "source.fixAll.eslint": "explicit" },
  "editor.tabSize": 2,
  "files.eol": "\n"
}
```

2. ESLint + Prettier basics (why + minimal setup)

- **Why:** ESLint catches *bugs and bad patterns* (unused vars, `==`, missing `await`); Prettier enforces *consistent formatting* so you never argue about style. They cover different jobs — use both.
- **Minimal setup** in your project folder:

```
npm init -y
npm install --save-dev eslint prettier
npx eslint --init          # answer the prompts (JS, modules, Node/browser)
```

- Add a Prettier config `.prettierrc` :

```
{ "singleQuote": true, "semi": true, "printWidth": 100 }
```

- Handy scripts in `package.json` :

```
"scripts": {  
  "lint": "eslint .",  
  "format": "prettier --write ."  
}
```

Rule of thumb: let Prettier own formatting, let ESLint own correctness. Don't fight formatOnSave.

3. Folder structure for today's project

Keep it small and clean — this is a JS-only sandbox:

```
day1-modern-js/  
├─ .gitignore  
├─ .prettierrc  
├─ package.json  
├─ src/  
|   ├─ 01-variables.js  
|   ├─ 02-arrow-functions.js  
|   ├─ 03-destructuring-spread.js  
|   ├─ 04-array-methods.js  
|   ├─ 05-promises-async.js  
|   ├─ utils.js           # your named + default exports live here  
|   └─ index.js           # imports from utils.js, runs the demos
```

One file per topic keeps reps isolated and easy to re-run.

4. Git commit practice

```
git init
```

Create `.gitignore` (Node):

```
node_modules/  
dist/  
.env  
*.log  
.DS_Store
```


Stage and commit with a meaningful message:

```
git add .  
git commit -m "Day 1: modern JS ES6+ practice - variables, arrows, destructuring, array  
methods, async/await, ES modules"
```

Good message habits: present tense, say *what* and *why*, keep the summary line short. Commit at logical checkpoints (e.g. after each topic file), not one giant end-of-day blob.

5. KnockoutJS vs React thinking — mindset primer (prep for Day 3)

You don't write React today, but start rewiring how you think:

- **Observables vs plain state:** Knockout `ko.observable` / `ko.computed` track dependencies and auto-update the UI (two-way binding). React has **no observables** — state is plain values; when it changes you tell React and it *re-renders*. Today's array methods returning *new* values (instead of mutating) is exactly this "recompute, don't observe" habit.
- **Manual DOM vs declarative UI:** Knockout `data-bind` + manual `ko.applyBindings` (and jQuery-style DOM tweaks) push you toward imperative "find element, change it." React is **declarative**: you describe what the UI *should look like* for a given state, and it figures out the DOM changes.
- **One-way data flow:** Knockout two-way binding syncs view↔model automatically. React data flows **one way** (parent → child via props); changes flow back up via callbacks. Predictable, easier to trace.
- **Immutable state:** The spread/ `map` / `filter` immutability you practiced today is not optional style in React — it's how React detects change. Treat "never mutate, always return a new object/array" as a core reflex starting now.

6. Browser DevTools for debugging JS

- **Console tab:** run snippets live, inspect variables, use `console.log`, `console.table(arr)` for arrays of objects, and `console.dir(obj)` for structure.
- **Sources tab:** set **breakpoints**, step over/into, and inspect scope + `this` at a pause — far better than scattering `console.log`.
- **debugger;** **statement:** drop it in code; execution pauses there when DevTools is open.
- **Network tab:** watch your Promise/ `fetch` calls, see pending vs resolved, status codes, and payloads — essential for the async section.
- Tip: expand a logged Promise in the console to confirm `pending` vs `fulfilled` and see what forgetting `await` actually looks like.

2

Detailed Notes — Read Once, Understand Everything

Read this once, slowly, from top to bottom. By the end you will understand everything from today. Every new word is explained in plain English right next to it. There is no hurry.

The big idea first

Modern JavaScript (people call it "ES6+") is just the newer, cleaner way to write JavaScript. You already know how to code, so you are not learning programming again — you are only learning nicer ways to write the same things.

There is one new habit to build today, and it matters more than any single feature:

Do not change data in place. Make a new copy instead.

This habit is called *immutability*. It feels strange at first (in KnockoutJS you happily changed things directly), but it is exactly how React works later. Build this habit now and Day 3 onwards becomes easy.

1. `let` and `const` — how you make a variable

- Use `const` for almost everything. It means: "I will not point this name at a different value later."
- Use `let` only when you truly need to change the value later (like a counter that goes up).
- **Never** use the old `var`.

One thing that confuses everyone at first: `const` stops you from *replacing* the value, but if the value is an object or an array, you can still change what is *inside* it.

```
const user = { age: 30 };  
user.age = 31;      // ✅ fine - we changed a field inside  
// user = {};      // ❌ error - we tried to replace the whole thing
```

Variables made with `let` / `const` only live inside the nearest pair of curly braces `{ }`. This is called *block scope*, and it is much safer than the old `var`.

Coming from C#: `const` is like a `readonly` reference — the field can't point somewhere new, but the object it points to can still change.

2. Arrow functions — a shorter way to write functions

An arrow function is just a shorter way to write a function. The `⇒` symbol is called a "fat arrow".

```
const add = (a, b) => a + b;    // if the body is one line, it returns automatically
const square = x => x * x;      // one input? you can skip the ()
```

If you use curly braces, you must write `return` yourself:

```
const greet = () => {
  return "hi";    // needed - braces do not auto-return
};
```

The one real difference from a normal function: an arrow function does **not** have its own `this`. It borrows `this` from the code around it. You don't need to fully understand `this` today — just know that arrow functions are the safe, normal choice for small callbacks (the little functions you pass to `map`, `filter`, `setTimeout`, etc.).

Coming from Knockout: this quietly removes the old `var self = this;` trick you used in view models.

3. Template literals — easy strings

Instead of gluing strings together with `+`, use backticks ``` and drop values in with `${ }`.

```
const name = "Het";
const count = 3;
const message = `Hello ${name}, you have ${count} items.`;
```

Bonus: text inside backticks can span multiple lines without any tricks.

Coming from C#: this is exactly `$"Hello {name}"` .

4. Destructuring — pulling values out neatly

Destructuring is a short way to take values out of an object or array and give them names.

```
const user = { id: 1, name: "Het", role: "dev" };
const { name, role } = user;    // now you have `name` and `role`
```

You can rename and set a fallback value at the same time:

```
const { role: job = "guest" } = user;    // job = "dev" (or "guest" if role was missing)
```

Arrays work by position:

```
const [first, second] = [10, 20];    // first = 10, second = 20
```

You will use this constantly in React to pull apart "props", so it's worth getting comfortable now.

5. Spread and rest — the `...` (three dots)

The three dots `...` do two opposite jobs. Which job depends on where you write it.

Spread = "open up" an array or object. Great for making a copy or joining things:

```
const copy = { ...user }; // a fresh copy of user
const withEmail = { ...user, email: "a@b.com" }; // copy + add a field
const joined = [...[1, 2], ...[3, 4]]; // [1, 2, 3, 4]
```

Rest = "gather the leftovers" into one array:

```
const [head, ...tail] = [1, 2, 3, 4]; // head = 1, tail = [2, 3, 4]
const sum = (...numbers) => numbers.reduce((t, n) => t + n, 0); // any number of inputs
```

Simple way to remember: opening something that already exists = **spread**; collecting the rest into something new = **rest**.

One caution: spread makes a *shallow* copy. It copies the top level, but nested objects inside are still shared.

6. Array methods — working with lists the modern way

These replace most of your old `for` loops. Each one takes a small function and does something with every item. This is the same idea as LINQ in C#.

- **map** — change every item, get a new array of the same size.

```
[1, 2, 3].map(n => n * 2); // [2, 4, 6] (like LINQ Select)
```

- **filter** — keep only the items that pass a test.

```
[1, 2, 3, 4].filter(n => n % 2 === 0); // [2, 4] (like LINQ Where)
```

- **reduce** — squeeze a whole list down to one value (a total, for example). Always give it a starting value.

```
[10, 20, 30].reduce((total, n) => total + n, 0); // 60 (like LINQ Aggregate)
```

- **find** — get the first item that matches (or `undefined` if none).

```
users.find(u => u.id === 5); // like LINQ FirstOrDefault
```

- **some** — returns **true** if *any* item matches. **every** — **true** if *all* match. (Like LINQ **Any** / **All**.)
- **sort** — puts items in order. **Careful:** **sort** changes the original array. Copy it first, and give it a compare function for numbers:

```
const sorted = [...nums].sort((a, b) => a - b); // small to big
```

Good news: **map**, **filter**, and **reduce** do **not** change the original — they hand you a new array. That fits perfectly with our "make a copy" habit.

7. Immutability — the habit that makes React easy

This deserves its own note because it is the mindset shift from KnockoutJS.

In Knockout you changed a list directly: **list.push(item)** or **list.remove(item)**. In modern JS (and React), you instead build a **new** list and use that.

```
// add      → make a new array with the extra item
const added = [...employees, newEmployee];

// update   → map, changing only the matching one
const updated = employees.map(e =>
  e.id === 3 ? { ...e, salary: 90000 } : e
);

// delete   → filter out the one you don't want
const removed = employees.filter(e => e.id !== 3);
```

Notice you never touched the old array — you made a new one each time. React uses this to notice "something changed" and update the screen. For now, just practice always returning a fresh copy.

8. Promises — a value that arrives later

Some jobs take time (like calling a server). A **Promise** is JavaScript's way of saying "I don't have the answer yet, but I will soon."

A Promise is in one of three states:

- **pending** — still waiting
- **fulfilled** — it worked, here is the value
- **rejected** — it failed, here is the error

Once it finishes, it never changes again.

```
fetchUser()
  .then(user => console.log(user.name)) // runs if it worked
  .catch(error => console.log(error.message)); // runs if it failed
```

Coming from C#: a Promise is basically a `Task<T>`.

9. `async` / `await` — the easy way to use Promises

`.then()` chains get messy. `async` / `await` lets you write waiting-code that reads top to bottom like normal code.

- Put `async` in front of a function to unlock `await` inside it.
- Put `await` in front of a Promise to say "pause here until the answer comes, then give me the value."
- Wrap it in `try` / `catch` to handle errors.

```
async function load() {
  try {
    const user = await fetchUser(); // waits, then continues
    console.log(user.name);
  } catch (error) {
    console.log("Something went wrong:", error.message);
  }
}
```

Coming from C#: this is almost identical to `async` / `await` there, so it will feel familiar fast.

Tip: if you have two independent calls, `await Promise.all([callA(), callB()])` runs them at the same time instead of one after the other.

10. ES modules — splitting code across files

Modules let you keep code in separate files and share pieces between them.

- `export` makes something available to other files.
- `import` pulls it in.

There are two kinds of export:

- **Named** exports — a file can have many. You import them by their exact name, inside `{ }`.
- **Default** export — a file can have only one. You import it without braces and can call it anything.

```
// math.js
export const add = (a, b) => a + b;          // named
export default function multiply(a, b) {    // default
  return a * b;
}

// app.js
import multiply, { add } from "./math.js"; // default first, then named in braces
```

Coming from C#: `import` / `export` are like `using` plus deciding exactly what each file shares.

11. Two small habits that save hours

- **Always compare with `===` (three equals), never `==` (two equals).** Two equals does surprising type conversions (`0 == ""` is `true` !). Three equals checks properly.
- **Prefer `??` over `||` for defaults** when `0`, `""`, or `false` are valid values. `value ?? "default"` only falls back when `value` is `null` or `undefined`.

One-line summary of the whole day

Write modern JS with `const`, arrow functions, template strings, and destructuring; work with lists using `map` / `filter` / `reduce` instead of loops; **always make a new copy instead of changing data in place**; and handle slow work with Promises and `async` / `await`. That's Day 1.

3 Common Mistakes to Avoid

These are the small slip-ups almost everyone makes on Day 1. Read them once now, and again during your practice — spotting them early saves a lot of confused debugging.

1. Changing data instead of copying it

Spread (`...`) only copies the top level, and `sort`, `push`, and `splice` change the original array.

- ✗ You sort a list and later find the original got reordered too.
- ✓ Copy first: `const sorted = [...list].sort(...)`. Build new arrays/objects instead of editing the old ones.

2. Forgetting `sort` needs a compare function for numbers

By default, `sort` treats everything as text.

- ✗ `[10, 2, 1].sort()` gives `[1, 10, 2]` — wrong for numbers.
- ✓ `[10, 2, 1].sort((a, b) => a - b)` gives `[1, 2, 10]`.

3. Forgetting `return` inside an arrow function with `{ }`

- ✗ `const double = x => { x * 2 };` returns nothing (`undefined`).
- ✓ Either drop the braces: `x => x * 2`, or write `return`: `x => { return x * 2; }`.

4. Forgetting `await`

Without `await`, you get the Promise itself, not the value inside it.

- ✗ `const user = fetchUser();` → `user` is a Promise, and later code breaks.
- ✓ `const user = await fetchUser();` (and make sure the function is `async`).

5. Not wrapping `await` in `try / catch`

If a Promise fails and nothing catches it, your program errors out.

- ✓ Put `await` calls inside `try { ... } catch (e) { ... }`, just like you would in C#.

6. Using `reduce` without a starting value

- ✗ `arr.reduce((a, b) => a + b)` crashes on an empty list.
- ✓ Always give the start value: `arr.reduce((a, b) => a + b, 0)`.

7. Using `==` instead of `===`

Two equals does weird conversions (`0 == ""` is `true`).

- ✓ Always use `===` and `!==`.

8. Thinking `const` means "cannot change at all"

`const` only stops you from replacing the whole value.

- `const list = []; list.push(1);` is perfectly fine — you changed what's inside, not the variable itself.

9. Mixing up named and default imports

- Named export → import with braces: `import { add } from './math.js'`.
- Default export → import without braces: `import multiply from './math.js'`.
- Getting this wrong usually gives you `undefined` and a confusing error.

10. Picking the wrong list method

- `find` gives you **one item**.
- `filter` gives you **an array**.
- `some` gives you **true/false**.
- Grabbing `filter` when you meant `find` (or the other way round) leads to bugs like treating an array as a single object.

4 Concept Notes — Recall & Interview (copy by pen)

This section is for **concept recall + interview prep**. Copy it by pen to memorize. Coming from KnockoutJS / .NET (C#) → React + TypeScript, so watch the one-line C# comparison bullets.

1. Variables & Scope

var vs let vs const

Definition:

- `var` = function-scoped, `let` / `const` = block-scoped (live only inside `{ }`).
- `const` = can't reassign the variable; `let` = can reassign.

✓ Advantages:

- `let` / `const` avoid accidental bugs by staying inside their block.
- `const` shows intent — "this won't be reassigned."

✗ Disadvantages / watch-outs:

- `var` is **hoisted** and initialized as `undefined` — leads to sneaky bugs.
- `let` / `const` are hoisted too, but sit in the **TDZ** (Temporal Dead Zone) — using them before declaration throws an error.
- `const` on an object still lets you change its **contents** (only the binding is locked).

💡 Interview tip:

- "Explain hoisting + TDZ" is a classic — say `var` = undefined, `let` / `const` = error before declaration.

📄 Example:

- `const x = 1; x = 2; // Error` — reassigning a const throws.
- C# note: `const` / `let` behave like normal C# block-scoped locals; `var` is the odd one out.

2. Functions

Arrow functions (lexical this)

Definition:

- Shorter function syntax: `(a, b) ⇒ a + b`.
- They **don't** get their own `this` — they borrow `this` from where they were defined (lexical `this`).

✓ Advantages:

- No more `const self = this` hacks — `this` just works inside callbacks.

- Cleaner one-liners for `map` / `filter`.

✗ Disadvantages / watch-outs:

- Can't be used as constructors (`new` fails).
- No own `arguments` object.
- Don't use as object methods if you need `this` = the object.

💡 Interview tip:

- "Difference between arrow and normal function?" → answer: `this` binding + not usable with `new`.

📄 Example:

- `const add = (a, b) => a + b;` — implicit return, no `function` keyword.
- C# note: like a C# lambda `(a, b) => a + b`, and lexical `this` is like how C# lambdas capture the enclosing `this`.

Higher-order functions

Definition:

- A function that **takes a function as an argument** or **returns a function**.

✅ Advantages:

- Enables reusable logic (`map`, `filter`, event handlers, middleware).
- Core to React (components receiving callback props).

📄 Example:

- `[1, 2, 3].map(n => n * 2)` — `map` is higher-order (it takes a function).
- C# note: like passing a `Func<>` / `Action<>` delegate or a LINQ lambda.

Closures

Definition:

- A function that **remembers variables** from the scope where it was created, even after that scope has finished.

✅ Advantages:

- Enables private/persistent state (counters, hooks like `useState` rely on this).

✗ Disadvantages / watch-outs:

- Can accidentally hold memory alive (leaks) if closures keep large data.
- Classic bug: `var` inside a loop shares one variable — use `let`.

💡 Interview tip:

- "What is a closure?" → "A function + the variables it closed over." Expect a loop question with `var` vs `let`.

Example:

- `function counter(){let c=0; return ()=>++c;}` — returned function keeps `c` alive.

The "this" keyword (basics)

Definition:

- `this` = the object that is **calling** the function; its value is decided at **call time**, not where it's written.

✓ Advantages:

- Lets one method work across different objects.

✗ Disadvantages / watch-outs:

- Rules to remember (what decides `this`):
 - Called as `obj.method()` → `this` = `obj`.
 - Called plain `fn()` → `this` = `undefined` (strict) or global.
 - Arrow function → `this` = borrowed from outside (lexical).
 - `call` / `apply` / `bind` → you set `this` manually.

💡 Interview tip:

- Very common: "What is `this` here?" — trace HOW the function was called.

Example:

- `obj.greet()` → inside, `this ≡ obj`.
- C# note: unlike C# where `this` is always the instance, JS `this` changes based on the call site.

3. Syntax Sugar (mostly Definition + Example)

Template literals

Definition:

- Strings with backticks that allow `${expression}` interpolation and multi-line text.

Example:

- ``Hello ${name}, age ${age}`` — embeds variables directly.
- C# note: like C# `$"Hello {name}"` interpolated strings.

Destructuring

Definition:

- Pull values out of objects/arrays into variables in one line.

Example:

- `const { name, age } = user;` — grabs two properties at once.

- `const [first, second] = arr;` — grabs by position.
- C# note: similar to C# tuple deconstruction `var (a, b) = point;`.

Spread vs rest

Definition:

- Same `...` symbol, opposite jobs.
- **Spread** = expand/copy items out. **Rest** = collect many items into one.

✗ Disadvantages / watch-outs:

- Spread makes only a **shallow** copy (nested objects still shared).

📄 Example:

- Spread: `const copy = [...arr];` — copies array items.
- Rest: `function sum(...nums){}` — gathers all args into `nums`.
- C# note: rest params are like C# `params object[] args`.

Ternary operator (?:)

Definition:

- One-line if/else: `condition ? valueIfTrue : valueIfFalse`.

✓ Advantages:

- Great for inline choices (heavily used in JSX rendering).

✗ Disadvantages / watch-outs:

- Don't nest deeply — becomes unreadable.

📄 Example:

- `const label = isOn ? "On" : "Off";`
- C# note: identical to C# `cond ? a : b`.

Optional chaining (?.) + nullish

Definition:

- `?.` safely reads deep properties — returns `undefined` instead of crashing if something is `null` / `undefined`.
- Often paired with `??` to give a fallback.

✓ Advantages:

- Kills "cannot read property of undefined" errors.

📄 Example:

- `user?.address?.city ?? "N/A"` — safe read with a default.
- C# note: same idea as C# `user?.Address?.City ?? "N/A"`.

4. Data & Comparison

Primitive vs reference types (+ shallow vs deep copy)

Definition:

- **Primitives** (string, number, boolean, null, undefined, symbol, bigint) are copied **by value**.
- **Objects/arrays/functions** are copied **by reference** (the variable holds a pointer).

✗ Disadvantages / watch-outs:

- Copying an object variable copies the **reference**, not the data — both point to the same thing.
- **Shallow copy** (`{ ... obj}`) copies top level only; nested objects are still shared.
- **Deep copy** (`structuredClone(obj)`) copies everything, fully independent.

💡 Interview tip:

- "Value vs reference types" + "how to deep clone" is a very common JS interview question.

📄 Example:

- `const b = a; b.x = 9; // a.x also 9` — reference shared.
- C# note: exactly like C# **struct (value)** vs **class (reference)** semantics.

== VS ===

Definition:

- `==` compares after **type coercion** (converts types). `===` compares value **and** type — no coercion.

✅ Advantages:

- `===` is predictable and safe.

✗ Disadvantages / watch-outs:

- `==` gives surprises: `0 == ""`, `null == undefined` are `true`.

💡 Interview tip:

- "Difference between == and ===?" — always answer: **use === by default**.

📄 Example:

- `0 == "" // true` but `0 === "" // false`.
- C# note: C# `==` doesn't auto-coerce like this; JS `==` is looser — avoid it.

?? vs ||

Definition:

- `||` returns the right side if the left is **any falsy** value.
- `??` returns the right side **only** if the left is `null` or `undefined`.

✗ Disadvantages / watch-outs:

- `0 || 5` gives `5` (bug!); `0 ?? 5` gives `0` (correct when 0 is valid).

💡 Interview tip:

- "When to use `??` over `||`?" → when `0`, `""`, or `false` are valid values you want to keep.

📄 Example:

- `const count = input ?? 0;` — keeps a real `0`, only defaults on null/undefined.

Truthy / falsy values

Definition:

- In a boolean context, values are treated as true or false.
- **Falsy** values (memorize): `false`, `0`, `""`, `null`, `undefined`, `NaN`. Everything else is **truthy**.

❌ Disadvantages / watch-outs:

- `"0"` (string) and `[]` (empty array) are **truthy** — surprising.

💡 Interview tip:

- "List all falsy values" is a frequent quick-fire question.

📄 Example:

- `if (name) { ... }` — runs only if `name` is truthy (not `""`, `null`, `undefined`).

5. Array Methods (with LINQ comparison)

map / filter / reduce / find / some / every / sort

Definition:

- Built-in ways to transform arrays without manual loops — each takes a callback.
- Quick meaning:
 - `map` → transform every item (LINQ **Select**).
 - `filter` → keep matching items (LINQ **Where**).
 - `reduce` → boil array down to one value (LINQ **Aggregate**).
 - `find` → first match (LINQ **FirstOrDefault**).
 - `some` → any match? (LINQ **Any**).
 - `every` → all match? (LINQ **All**).
 - `sort` → order items (LINQ **OrderBy**).

✅ Advantages:

- Readable, chainable, declarative — no index bookkeeping.

❌ Disadvantages / watch-outs:

- `sort` **mutates** the original array AND sorts as strings by default — pass a comparator: `sort((a,b)⇒a-b)`.
- `map` / `filter` return **new** arrays (good for immutability); a plain loop with side effects doesn't.

💡 **Interview tip:**

- "Implement/explain reduce" is common — it takes an accumulator + current item.

📄 **Example:**

- `arr.filter(x ⇒ x > 2).map(x ⇒ x * 10)` — chain like LINQ `.Where().Select()`.
- C# note: these are JS's LINQ; biggest gotcha vs C# is `sort` mutating + string-sorting.

Immutability (make-a-copy habit)

Definition:

- Never change existing data — create a **new** copy with the change instead.

✅ **Advantages:**

- React relies on new references to detect changes and re-render.
- Prevents shared-state bugs.

❌ **Disadvantages / watch-outs:**

- Spread copies are **shallow** — nested objects need their own copy too.

💡 **Interview tip:**

- "Why immutability in React?" → new reference = React knows to re-render.

📄 **Example:**

- `const next = { ...user, age: 30};` — new object, original untouched.
- C# note: like using a `record` with `with { age = 30 }` instead of mutating.

6. Async JavaScript

The event loop (beginner level)

Definition:

- JS runs on a **single thread** but stays non-blocking.
- Slow tasks (timers, network) are handed off; when done, their callbacks **queue up** and run only when the main call stack is empty.

✅ **Advantages:**

- One thread handles async work without you managing threads/locks.

❌ **Disadvantages / watch-outs:**

- Heavy synchronous code **blocks everything** (UI freezes) — nothing else runs meanwhile.

💡 Interview tip:

- "How does single-threaded JS do async?" → call stack + callback queue + event loop.

📄 Example:

- `setTimeout(fn, 0)` still runs `fn` **after** current code finishes — proof of the queue.
- C# note: no `Thread` / `Task` parallelism here by default; it's cooperative, like a single-threaded `async` context.

Promises

Definition:

- An object representing a future value: **pending** → **fulfilled** or **rejected**.
- Handle with `.then()` (success) and `.catch()` (error).

✅ Advantages:

- Avoids nested callback hell; chainable.

❌ Disadvantages / watch-outs:

- Forgetting `.catch()` = silent/unhandled errors.

📄 Example:

- `fetch(url).then(r => r.json()).catch(err => ...)` — chained handling.
- C# note: a Promise is like a C# `Task<T>`.

async / await (+ try/catch)

Definition:

- `async` marks a function that returns a Promise; `await` pauses until a Promise resolves — reads like normal top-down code.
- Wrap `await` in `try/catch` to handle errors.

✅ Advantages:

- Cleaner and more readable than `.then()` chains.

❌ Disadvantages / watch-outs:

- `await` only works inside an `async` function.
- Awaiting things one-by-one in a loop can be slow — use `Promise.all` for parallel.

💡 Interview tip:

- "async/await vs Promises?" → same thing under the hood, just nicer syntax.

📄 Example:

- `try { const r = await fetch(url); } catch(e) {}` — await + error handling.
- C# note: nearly identical to C# `async` / `await` with `try/catch`.

7. Modules

ES modules (named vs default export)

Definition:

- Split code across files with `export` / `import`.
- **Named export** = many per file, import by exact name in `{ }`.
- **Default export** = one per file, import with any name (no braces).

✅ Advantages:

- Clear dependencies; supports tree-shaking (drops unused code).

❌ Disadvantages / watch-outs:

- Named imports must match the name (and braces); default import can be renamed freely — don't confuse the two syntaxes.

📄 Example:

- `export const x = 1;` → `import { x } from './f';` (named).
- `export default App;` → `import App from './App';` (default).
- C# note: rough analog of C# `namespace` + `using`, but per-file and explicit per symbol.